

CONFIDENTIAL SECURITY REPORT

Security Audit

Lucky Squares Australia

4 Critical

3 High

5 Medium

3 Low

Date **16 May 2026**

Application **Lucky Squares Australia v0.1.0**

Stack **Next.js 15 · Supabase (Postgres + RLS + Realtime) · React 19**

Scope **Full codebase review — client, server RPCs, RLS policies, config**

This report is confidential. Do not distribute outside the Lucky Squares Australia team.

Executive Summary

A read-only security review was conducted across the full Lucky Squares Australia codebase, including the Next.js front-end, Supabase schema and RLS policies, and all client-side data handling flows. Fifteen findings were identified across four severity tiers.

The four Critical findings pose the most immediate risk and should be addressed before the platform accepts real fundraisers or real money. In particular, organiser bank details are currently readable by any anonymous user via the Supabase API, and the square claiming flow marks purchases as complete before any payment has been received or verified.

The High and Medium findings are hardening improvements — none represent an immediate existential risk, but they should be addressed in the weeks following launch. Low findings are minor issues or best-practice gaps that can be addressed incrementally.

No changes have been made to the codebase as part of this review. All findings are listed below for prioritised remediation.

#	Finding	Severity	Primary File
1	Bank details exposed to anonymous users	Critical	schema.sql, f/[id]/page.js
2	Missing RPC implementations	Critical	schema.sql, LiveGrid.js
3	No server-side square count limit	Critical	LiveGrid.js
4	Squares claimed before payment verified	Critical	LiveGrid.js
5	No rate limiting on reservations	High	LiveGrid.js, schema.sql
6	Buyer name not validated server-side	High	LiveGrid.js, schema.sql
7	No CSRF protection on RPC calls	High	LiveGrid.js
8	No HTTP security headers	Medium	next.config.mjs
9	RLS does not validate reserved_until	Medium	schema.sql
10	No server-side email validation	Medium	LiveGrid.js
11	No audit trail for sensitive operations	Medium	schema.sql

#	Finding	Severity	Primary File
12	No length limits on campaign fields	Medium	FundraiseApp.js
13	No numeric validation on price field	Low	FundraiseApp.js
14	No 2FA for organiser accounts	Low	FundraiseApp.js
15	No fraud pattern detection	Low	Platform-wide

Findings

Critical

1

Bank details exposed to anonymous users

The RLS policy on the fundraisers table grants *any unauthenticated user* read access to every column of active or drawn fundraisers. This includes `bank_account_name`, `bank_bsb`, `bank_account`, and `stripe_account_id`. Anyone who knows (or guesses) a fundraiser UUID can query the Supabase REST API directly and retrieve the organiser's full banking credentials — no UI, no authentication required.

This also means bank details are transmitted to every visitor of the public fundraiser page, even those who never intend to buy a square.

`supabase/schema.sql` lines 69–70

`src/app/f/[id]/page.js` line 46

`src/components/app/LiveGrid.js` lines 269–284

2

Missing RPC implementations

The application calls four Postgres RPC functions that are not defined anywhere in `schema.sql`: `reserve_square`, `release_square`, `claim_squares`, and `create_fundraiser_squares`. The schema only defines `handle_new_user`, `record_draw`, and `expire_stale_reservations`.

If these RPCs have not been created directly in the Supabase dashboard without validation logic, every square interaction in the live app is either silently failing or operating without server-side guards — meaning no atomicity, no limit checks, and no ownership verification at the database level.

`supabase/schema.sql`

`src/components/app/LiveGrid.js` lines 134, 183, 188, 199

`src/components/app/FundraiseApp.js` line 940

3

No server-side square count limit

The 10-square-per-buyer cap is enforced only in client-side JavaScript (`const MAX_CART = 10`). The `claim_squares` RPC is called with a client-supplied array of square numbers. There is no server-side check on this array's length.

A malicious buyer can call the RPC directly — bypassing the browser entirely — and pass an array of 100 square numbers, claiming an entire grid in a single request. The limit must be enforced inside the RPC itself using a server-side guard.

```
src/components/app/LiveGrid.js line 9 (MAX_CART)
```

```
src/components/app/LiveGrid.js line 199 (claim_squares call)
```

4

Squares marked as claimed before payment is received

The `handlePay` function immediately writes squares as claimed in the database and displays the success screen, then instructs the buyer to go and make the bank transfer or pay in person. There is no mechanism to verify payment was received before the claim is recorded, no way to cancel a non-paying registration, and no payment status field on the squares table.

A buyer can claim any number of squares across any fundraiser and simply never pay. The organiser sees a "sold" grid but receives no funds. The squares are blocked from legitimate buyers indefinitely. This is particularly critical given the addition of the in-person payment option, where there is no technical link between clicking "Confirm squares" and actual payment occurring.

```
src/components/app/LiveGrid.js lines 196-203
```

5**No rate limiting on square reservation calls**

There is no throttle on calls to `reserve_square` or `release_square`. A script can cycle reserve and release on every available square at high speed, flooding the Realtime channel with bogus updates, making the grid unusable for legitimate buyers, and potentially exhausting Supabase request quotas — creating unexpected costs for the organiser.

```
src/components/app/LiveGrid.js line 178
```

```
supabase/schema.sql (no rate limit policy)
```

6**Buyer name not validated server-side**

Buyer names are stored in the database with no length cap or character restriction. React's JSX auto-escaping protects the grid view from XSS when names are rendered in the browser. However, if buyer names are used in any non-React context — emails, CSV exports, PDF generation, admin dashboards, or future integrations — they will be unescaped and a name like `<script>...</script>` could execute arbitrary code.

Additionally, with no length limit, a buyer could submit an extremely long name, causing UI layout issues or database bloat.

```
src/components/app/LiveGrid.js line 202
```

```
supabase/schema.sql line 104 (no constraint on buyer_name)
```

7

No CSRF protection on RPC calls

All state-mutating RPC calls are made directly from client-side JavaScript with no CSRF token. The Supabase anon key is intentionally public (required for the public grid to load), which means any third-party website can trigger reservation and release calls on behalf of a visitor. A malicious page could embed scripts that spam the reservation API using a visiting user's session or anonymously, disrupting active fundraisers.

```
src/lib/supabase/client.js
```

```
src/components/app/LiveGrid.js (all RPC calls)
```

Medium

8

No HTTP security headers configured

`next.config.mjs` is essentially empty — no security headers are set. Missing headers include `Content-Security-Policy` (restricts which scripts can run), `X-Frame-Options` (prevents clickjacking via iframe embedding), `Strict-Transport-Security` (enforces HTTPS), and `X-Content-Type-Options: nosniff` (prevents MIME sniffing). Without these, the app is vulnerable to clickjacking and cannot restrict third-party script injection.

```
next.config.mjs
```

9

RLS policy does not validate the reserved_until timestamp

The RLS update policy on the `squares` table only checks that the status transitions from `available` to `reserved`. It does not validate the value of `reserved_until`. A direct API call could set a reservation expiry date years in the future, effectively locking a square unless the `expire_stale_reservations` cron job catches it — and only if that cron job is actually running.

```
supabase/schema.sql lines 124-127
```

10

No server-side email validation

Buyer email is accepted via an HTML5 `type="email"` input, which provides basic browser-level validation only. This can be bypassed by calling the API directly. No regex or format check exists in the schema or RPCs. Invalid or malicious strings can be stored and will break any future notification, receipt, or communication flow.

```
src/components/app/LiveGrid.js line 376
```

```
supabase/schema.sql (no email constraint)
```

11

No audit trail for sensitive operations

There is no logging of who claimed which squares, when draws were run, who accessed bank details, or payment status changes. Without an audit log, dispute resolution between organisers and buyers is impossible, fraud detection cannot be implemented, and the platform cannot demonstrate compliance with fundraising governance requirements in any Australian state.

```
supabase/schema.sql (no audit_log table)
```

12

No length limits on campaign input fields

Campaign title, description, organisation name, and thank-you message fields have no `maxLength` attribute and no server-side size constraint. A very large description (e.g. several megabytes) would be accepted by the database, transmitted to every visitor of the fundraiser page, and could cause the page to become unusable or crash.

```
src/components/app/FundraiseApp.js lines 556, 564, 568
```

Low

13

No server-side validation on price per square

The price input uses a client-side `min="1"` constraint. The database type will prevent the worst cases, but there is no RPC-level validation to reject a zero or negative price if the client-side guard is bypassed. An organiser could publish a fundraiser with a \$0 price per square.

```
src/components/app/FundraiseApp.js line 483
```

14

No two-factor authentication for organiser accounts

Organiser accounts control live fundraisers, live bank details, and the ability to run draws. Account takeover via a weak or reused password has outsized impact compared to a typical consumer app. Supabase supports TOTP-based 2FA but it is not currently enabled or enforced.

```
src/components/app/FundraiseApp.js (auth flows)
```

15

No fraud pattern detection

There are no alerts for unusual usage patterns: a single buyer claiming all squares in a fundraiser, one email address appearing across many fundraisers, or fundraisers that have never run a draw long after their scheduled date. Without detection, fraud and abuse can go unnoticed indefinitely.

```
Platform-wide
```

Findings Not Applicable

Fundraiser UUID enumeration — Fundraisers correctly use `gen_random_uuid()`, making enumeration impractical.

Draw ownership bypass — The `record_draw` RPC correctly validates `auth.uid()` against the fundraiser's `owner_id` server-side. The UI restriction (`isOwner` flag) is defence-in-depth only.

Auth token expiry — The Supabase JS SDK handles token refresh automatically. No manual handling is required.

Remediation Recommendations

Grouped by priority. Items marked **Before Live** should be resolved before the platform accepts real money or real fundraisers. Items marked **Soon After** are hardening improvements for the weeks following initial launch. Items marked **Ongoing** are best-practice additions that can be built incrementally.

Before Go-Live (Criticals)

- 1 Create a restricted database view (or update the RLS select policy) so that `bank_bsb`, `bank_account`, `bank_account_name`, and `stripe_account_id` are never returned in public queries. Return bank details only at checkout — ideally via a server-side route that verifies the buyer has an active reservation.
- 2 Define all four missing RPCs (`reserve_square`, `release_square`, `claim_squares`, `create_fundraiser_squares`) as security definer functions in the schema. Each must enforce atomicity, validate inputs, and hard-cap the square count at 10 per transaction server-side.
- 3 Introduce a `payment_status` field on the `squares` table (`pending` | `confirmed` | `cancelled`). Squares should move to `pending` at checkout and only show as fully sold once the organiser marks payment as received (bank/in-person) or a Stripe webhook confirms capture.

Soon After Launch (High and Medium)

- 4 Add Supabase rate limiting on the reservation RPCs, or implement a short server-side cooldown using a `last_reserved_at` timestamp per IP or session.
- 5 Add a CHECK constraint on `buyer_name` in the schema: max 120 characters, printable characters only. Add server-side email format validation in the `claim_squares` RPC.
- 6 Add HTTP security headers in `next.config.mjs`: `Strict-Transport-Security`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, and a `Content-Security-Policy` scoped to Supabase and Google Fonts.
- 7 Strengthen the `squares` RLS update policy to validate that `reserved_until` is between `now() + 5 minutes` and `now() + 15 minutes`.
- 8 Create an `audit_log` table recording claims, releases, draws, and payment status changes with `actor`, `fundraiser_id`, `action`, `details`, and `created_at`.
- 9 Add `maxLength` attributes and corresponding database CHECK constraints to all campaign text fields: title (120), organisation (120), description (2000), thank-you message (300).

Ongoing Improvements (Low)

- 10 Enable Supabase TOTP two-factor authentication and consider making it mandatory for organiser accounts.
- 11 Add server-side validation for `price_per_sq`: must be a positive number with a sensible upper bound (e.g. \$500 max).
- 12 Build a basic fraud monitoring dashboard: alert when one buyer claims more than 30% of a grid, or when a fundraiser has had no draw run more than 30 days after its scheduled date.
- 13 When Stripe integration is built, verify all payments via webhook before marking squares as sold. Never trust the client's success callback alone.

Audit conducted: 16 May 2026. Application version: Lucky Squares Australia v0.1.0. This is a read-only review — no changes were made to the codebase. All findings are based on static analysis of source files and schema. Testing in a live environment may reveal additional issues. This report should be treated as confidential and stored securely.